Projeto de Processador MIPS com Pipeline para Algoritmos de Ordenação em Hardware

1st José Dercy Vieira da Silva Filho
Engenharia da Computação (FEEC)
Universidade Federal do Amazonas (UFAM)
Manaus, Brasil
jose.filho@ufam.edu.br

2nd Vinicius Menezes
Engenharia da Computação (FEEC)
Universidade Federal do Amazonas (UFAM)
Manaus, Brasil
vinicius.menezes@ufam.edu.br

3rd Isabela Furtado
Engenharia da Computação (FEEC)
Universidade Federal do Amazonas (UFAM)
Manaus, Brasil
isabela.furtado@icomp.ufam.edu.br

I. Introdução

A evolução dos sistemas computacionais modernos apoia-se no uso de abstrações, técnica fundamental para simplificar projetos complexos ocultando detalhes de baixo nível. Nesse cenário, a arquitetura de conjunto de instruções, conhecida como ISA, estabelece a interface abstrata essencial entre o software e o hardware do processador. O conjunto de instruções MIPS destaca-se pela simplicidade e pela regularidade, características projetadas para facilitar a criação de compiladores e otimizar a implementação do circuito físico [1], [2].

Para alcançar maior desempenho em sistemas digitais, a técnica de processamento em pipeline divide a execução em múltiplos estágios, tipicamente busca, decodificação, execução, memória e escrita do resultado, aumentando significativamente a vazão de instruções executadas. Contudo, a organização em pipeline exige o tratamento de riscos no hardware — chamados de hazards de dados, de controle e estruturais —, que podem estagnar a execução do sistema caso não sejam resolvidos.

Neste contexto, o presente trabalho descreve o projeto e a implementação modular de um processador MIPS em pipeline usando a linguagem de descrição de hardware Verilog. O objetivo principal do projeto consiste em construir uma arquitetura parametrizada que trate os hazards da execução. A validação do sistema será realizada por meio da execução de um algoritmo de ordenação, composto pelas funções sort e swap. Esta primeira entrega documenta o progresso inicial do grupo — abrangendo a especificação da arquitetura, o planejamento no Trello e o mapeamento binário do código de programa.

II. Metodologia e Plataforma de Desenvolvimento

A metodologia adotada para a criação do processador MIPS baseia-se em um fluxo de projeto estruturado em camadas de abstração, integrando desde a compilação do software em nível de aplicação até a descrição estrutural do hardware. A arquitetura do caminho de dados e da unidade de controle adota uma abordagem modular e parametrizada — permitindo modificar a largura de barramentos e componentes de forma simples.

Para a especificação do circuito digital — cobrindo os cinco estágios do pipeline e as unidades de encaminhamento e detecção de hazards —, utiliza-se a linguagem de descrição de hardware Verilog HDL [6]. O conjunto de ferramentas de desenvolvimento organiza-se nas seguintes etapas funcionais:

- Tradução C para Assembly: O algoritmo de ordenação — composto pelas funções sort e swap — é traduzido manualmente para Assembly, respeitando a sintaxe do conjunto de instruções MIPS e os exemplos apresentados na literatura da disciplina [2].
- Teste e Verificação Funcional: O código é compilado e simulado no ambiente MARS [3] para inspeção comportamental. Um script auxiliar foi desenvolvido para executar a função sort e atestar visualmente a ordenação do vetor de saída.
- Geração e Validação do Código de Máquina: Utiliza-se o conjunto mips-linux-gnu-binutils [4] com a diretiva que proíbe a reordenação de instruções (.set noreorder). O binário gerado é comparado com o artefato produzido pelo MARS. Uma amostragem aleatória de 10 palavras do binário final (aproximadamente 20% do programa) foi verificada manualmente para confirmar a corretude da saída.
- Simulação e Verificação RTL: A codificação dos módulos em Verilog é realizada no ambiente VS Code e compilada via Icarus Verilog. A verificação do tempo de execução e dos sinais de controle do pipeline é feita inspecionando as formas de onda no GTKWave.
- Síntese e Prototipagem em Hardware: A validação física do processador será executada na placa de desenvolvimento FPGA DE10-Lite da Intel [5]. O

mapeamento lógico, o roteamento de pinos e a síntese final utilizam a ferramenta Intel Quartus Prime — combinada com o ModelSim para simulações temporais avançadas.

A. Justificativa da Escolha da FPGA para Prototipagem

A escolha da placa FPGA DE10-Lite justifica-se por três critérios técnicos e logísticos:

- Capacidade de Hardware: O dispositivo possui elementos lógicos e blocos de memória interna suficientes para comportar a síntese completa do caminho de dados e da unidade de controle do processador MIPS.
- Infraestrutura Institucional: A ampla disponibilidade deste modelo nos laboratórios da instituição garante acesso contínuo para a etapa de prototipagem física.
- Domínio do Ecossistema: A familiaridade prévia da equipe com a arquitetura da placa e com o fluxo de síntese do Intel Quartus Prime otimiza o cronograma de desenvolvimento e depuração.

III. Mapeamento do Código e Memória de Programa

A transição gradativa entre a lógica em alto nível descrita em C, a rotina em Assembly MIPS e a tradução final para a linguagem de máquina é estruturada nas subseções a seguir.

A. Descrição do Algoritmo de Ordenação

A lógica do algoritmo Bubble Sort, composto pelas funções sort e swap fornecidas na especificação do projeto, define o funcionamento dos laços aninhados e a troca de elementos adjacentes quando fora de ordem.

```c
void sort (int v[], int n) {
    int i, j;
    for (i = 0; i < n; i += 1) {
        for (j = i - 1; j >= 0 &&
         v[j] > v[j + 1]; j -= 1) {
            swap(v, j);
        }
    }
}
void swap (int v[], int k) {
    int temp;
    temp = v[k];
    v[k] = v[k+1];
    v[k+1] = temp;
}
```

Listing 1. Algoritmo de ordenação em Linguagem C.

B. Mapeamento do Procedimento Swap

O procedimento swap realiza a permutação entre dois elementos adjacentes do vetor na memória. Como não realiza chamadas a outras rotinas, classifica-se como um procedimento folha — dispensando a alocação de quadro de pilha para preservação do registrador $ra. Os parâmetros de entrada consistem no endereço base do vetor em $a0 e no índice k em $a1. A listagem 2 apresenta o mapeamento das instruções para a arquitetura MIPS.

```
sll  $t0, $a1, 2      # t0 = k * 4
add  $t0, $a0, $t0    # t0 = endereco base v + offset
lw   $t1, 0($t0)      # t1 = v[k]
lw   $t2, 4($t0)      # t2 = v[k+1]

sw   $t2, 0($t0)      # v[k] = t2
sw   $t1, 4($t0)      # v[k+1] = t1

jr   $ra              # retorna para o caller
```

Listing 2. Procedimento swap em Assembly MIPS.

O segmento de texto na arquitetura MIPS inicia-se no endereço-base 0x00400000, com alinhamento estrito de 4 bytes por instrução. A Tabela II apresenta o mapeamento da rotina swap na memória de programa, detalhando o endereço físico, a mnemônica Assembly, o formato de instrução e a respectiva codificação binária.

TABLE I
Memória de Programa e Mapeamento Binário

Endereço RAM/ROM	Instrução Assembly	Formato MIPS
0x00400000	sll $t0, $a1, 2	Formato-R
0x00400004	add $t0, $a0, $t0	Formato-R
0x00400008	lw $t1, 0($t0)	Formato-I
0x0040000c	lw $t2, 4($t0)	Formato-I
0x00400010	sw $t2, 0($t0)	Formato-I
0x00400014	sw $t1, 4($t0)	Formato-I
0x00400018	jr $ra	Formato-R

O código binário gerado pelo MARS segue:

TABLE II
Memória de Programa e Mapeamento Binário da Função Swap

Instrução Assembly	Instrução Binário gerado pelo MARS
sll $t0, $a1, 2	00000000000001010100000010000000
add $t0, $a0, $t0	00000000100010000100000000100000
lw $t1, 0($t0)	10001101000010010000000000000000
lw $t2, 4($t0)	10001101000010100000000000000100
sw $t2, 0($t0)	10101101000010100000000000000000
sw $t1, 4($t0)	10101101000010010000000000000100
jr $ra	00000011111000000000000000001000

Para exemplificar a decodificação no hardware, analisa-se a instrução lw $t1, 0($t0), representada pela palavra binária 10001101000010010000000000000000. Por pertencer ao formato tipo-I, o particionamento dos campos opera sob a seguinte convenção:

- opcode [31:26] — 100011 (35_{10} ou 0x23): identifica a operação de leitura de memória (load word), configurando a unidade de controle para ativar os sinais de leitura na memória de dados (MemRead) e escrita no banco de registradores (RegWrite).
- rs [25:21] — 01000 (8_{10}): mapeia o registrador-base $t0, cujo conteúdo fornece o endereço inicial a ser computado na ALU.
- rt [20:16] — 01001 (9_{10}): mapeia o registrador de destino $t1, que receberá a palavra lida da memória no estágio de escrita (write-back).

- imediato [15:0] — 0000000000000000 (0_{10}): representa o deslocamento (offset) em 16 bits, submetido à extensão de sinal antes da soma com o valor de rs.

C. Codificação da Memória de Programa

A memória de instrução é inicializada diretamente pelo código de máquina resultante da montagem. A Tabela III contém a representação em formato binário, omitindo as abstrações da linguagem de montagem.

TABLE III
Memória de Programa Completa em Binário

Instrução Assembly	Instrução Binário Gerado pelo MARS
addi $sp, $sp, -20	00100011101111101111111111101100
sw $ra, 16($sp)	10101111101111110000000000010000
sw $s3, 12($sp)	10101111101100110000000000001100
sw $s2, 8($sp)	10101111101100100000000000001000
sw $s1, 4($sp)	10101111101100010000000000000100
sw $s0, 0($sp)	10101111101100000000000000000000
add $s2, $a0, $zero	00000000100000001001000000100000
add $s3, $a1, $zero	00000000101000001001100000100000
add $s0, $zero, $zero	00000000000000001000000000100000
slt $t0, $s0, $s3	00000010000100110100000000101010
beq $t0, $zero, exit1	00010001000000000000000000010110
nop	00000000000000000000000000000000
addi $s1, $s0, -1	00100010000100011111111111111111
slti $t0, $s1, 0	00101010001010000000000000000000
bne $t0, $zero, exit2	00010101000000000000000000001111
nop	00000000000000000000000000000000
sll $t1, $s1, 2	00000000000100010100100010000000
add $t2, $s2, $t1	00000010010010010101000000100000
lw $t3, 0($t2)	10001101010010110000000000000000
lw $t4, 4($t2)	10001101010011000000000000000100
slt $t0, $t4, $t3	00000001100010110100000000101010
beq $t0, $zero, exit2	00010001000000000000000000001000
nop	00000000000000000000000000000000
add $a0, $s2, $zero	00000010010000000010000000100000
add $a1, $s1, $zero	00000010001000000010100000100000
jal swap	00001100000100000000000000101001
nop	00000000000000000000000000000000
addi $s1, $s1, -1	00100010001100011111111111111111
j for2tst	00001000000100000000000000001101
nop	00000000000000000000000000000000
addi $s0, $s0, 1	00100010000100000000000000000001
j for1tst	00001000000100000000000000001001
nop	00000000000000000000000000000000
lw $s0, 0($sp)	10001111101100000000000000000000
lw $s1, 4($sp)	10001111101100010000000000000100
lw $s2, 8($sp)	10001111101100100000000000001000
lw $s3, 12($sp)	10001111101100110000000000001100
lw $ra, 16($sp)	10001111101111110000000000010000
addi $sp, $sp, 20	00100011101111010000000000010100
jr $ra	00000011111000000000000000001000
nop	00000000000000000000000000000000
sll $t0, $a1, 2	00000000000001010100000010000000
add $t0, $a0, $t0	00000000100010000100000000100000
lw $t1, 0($t0)	10001101000010010000000000000000
lw $t2, 4($t0)	10001101000010100000000000000100
sw $t2, 0($t0)	10101101000010100000000000000000
sw $t1, 4($t0)	10101101000010010000000000000100
jr $ra	00000011111000000000000000001000
nop	00000000000000000000000000000000

IV. Planejamento e Gestão de Atividades

O planejamento inicial do projeto foi estruturado na plataforma Jira, segmentado em três épicos principais: Desenvolvimento Front-end do pipeline, Desenvolvimento Back-end e Validação em Software. A proposta original previa a codificação paralela dos módulos de hardware em Verilog e a tradução do software durante a primeira sprint, culminando na integração Top-Level e na configuração inicial da FPGA.

No entanto, durante o ciclo de desenvolvimento, a equipe identificou a necessidade de readequar o cronograma. A construção modular do hardware exige uma base robusta de testes para validação; especificamente, o desenvolvimento e teste dos estágios de Busca (IF) e Decodificação (ID) são inviáveis sem uma Memória de Instruções devidamente populada com o código do programa. Por esse motivo, as atividades foram repensadas, priorizando inteiramente o fluxo de software e a especificação funcional nesta primeira etapa.

A organização das atividades efetivamente executadas justificou-se da seguinte forma:

- Mapeamento C para Assembly MIPS: O esforço principal foi direcionado para a tradução do algoritmo Bubble Sort e da função Swap. Essa etapa foi estritamente necessária para garantir a alocação correta de variáveis (como v em $a0, n em $a1) e a eliminação de pseudo-instruções. Sem o domínio exato de quais instruções seriam utilizadas, a parametrização da Unidade de Controle no hardware seria baseada em conjecturas.
- Validação Funcional no MARS: A simulação prévia do Assembly isolou os erros de lógica do algoritmo dos possíveis erros físicos do processador.
- Geração do Código de Máquina: A etapa de montagem (conversão para binário/hexadecimal) tornou-se prioritária, pois os módulos de inicialização do Verilog (usando $readmemb ou $readmemh) dependem estritamente desse artefato para alimentar o Program Counter (PC).
- Especificação da Arquitetura: Definição dos fluxos de dados e preparação do ecossistema de compilação (Icarus Verilog e GTKWave).

A. Replanejamento e Backlog Futuro

Com a base de software estabelecida e o código de máquina validado, as atividades de codificação do hardware que compunham os Épicos 1 e 2 foram remanejadas para o backlog da próxima sprint no Jira/Trello. As atividades foram reestruturadas nos seguintes blocos lógicos:

- Módulos do Caminho de Dados (Datapath): Implementação sequencial e unitária do Banco de Registradores, ALU e Memória de Dados, com testes automatizados (testbenches) validados diretamente contra as instruções agora conhecidas do Bubble Sort.
- Registradores de Pipeline: Inserção das barreiras de segmentação (IF/ID, ID/EX, EX/MEM, MEM/WB).
- Unidade de Tratamento de Hazards: Tarefa crítica adicionada ao planejamento, necessária para a implementação de encaminhamento (forwarding) e bolhas

(stalls), visto que os laços do algoritmo de ordenação geram dependências de dados iminentes no pipeline.
- Integração Top-Level e Prototipagem FPGA: Conexão de todos os componentes, simulação final por formas de onda e mapeamento físico na placa DE10-Lite.

V. Considerações Finais

Esta primeira entrega documentou a fase inicial e o alicerce fundamental para a construção do processador MIPS com arquitetura em pipeline. O desenvolvimento concentrou-se na especificação funcional e na camada de software, com destaque para o mapeamento bem-sucedido do algoritmo de ordenação Bubble Sort e do procedimento Swap, desde a linguagem C até a geração final do código de máquina. A validação comportamental no simulador MARS atestou a corretude lógica da rotina, garantindo que as operações matemáticas, de acesso à memória e de desvios condicionais operem sem falhas.

A reestruturação do planejamento inicial, descrita na seção anterior, provou-se uma decisão metodológica acertada. Ao priorizar a extração e a validação das instruções binárias que irão compor a Memória de Programa, estabeleceu-se um ambiente de testes determinístico. O código de máquina extraído do MARS atuará como o prova para a verificação do hardware, permitindo que o desenvolvimento dos estágios do processador seja testado e depurado contra o binário real da aplicação, evitando abstrações ou dados fictícios.

Para as próximas etapas, os esforços retornarão à linguagem de descrição de hardware (Verilog). O principal desafio técnico poderá envolver a instanciação modular do caminho de dados (datapath) e a inserção cirúrgica dos registradores de barreira de segmentação. Adicionalmente, as interdependências de variáveis dentro dos laços aninhados do algoritmo de ordenação exigirão uma implementação rigorosa da unidade de detecção de hazards e do mecanismo de forwarding. A validação final do projeto dar-se-á com a integração completa dos módulos (Top-Level), verificação das formas de onda temporais no GTKWave e, por fim, a síntese física do sistema em silício utilizando a placa FPGA DE10-Lite.

References

[1] D. A. Patterson e J. L. Hennessy, Computer Organization and Design - The Hardware / Software Interface, 4ª ed. revisada, Academic Press, 2012.

[2] T. Brito, "Instrucoes: Linguagem do Computador," Notas de Aula, Universidade Federal do Amazonas, Manaus, 2026.

[3] P. Sanderson e K. Vollmar, "MARS – MIPS Assembler and Runtime Simulator," Missouri State University, 2014. [Online]. Disponível em: https://computerscience.missouristate.edu/mars-mips-simulator.htm

[4] Free Software Foundation, "GNU Binutils Manual," Free Software Foundation. [Online]. Disponível em: https://sourceware.org/binutils/docs/

[5] Universidade Federal do Amazonas (UFAM), Faculdade de Tecnologia, Departamento de Eletrônica e Computação, Super Roteiros: FPGA DE10-LITE, 2024.

[6] Institute of Electrical and Electronics Engineers, IEEE Standard for Verilog Hardware Description Language, IEEE Std 1364-2005, 2006. [Online]. Available: https://standards.ieee.org/